

形式化方法 大作业实验2 使用K框架构建Orc的语义模型

PB22111599 杨映川

参照论文来自CCF-A会议FME(International Symposium on Formal Methods)
2015年的文章 *Towards Formal Verification of Orchestration Computations Using the K Framework*

20th FM 2015: Oslo, Norway

    Nikolaj S. Bjørner, Frank S. de Boer:
FM 2015: Formal Methods - 20th International Symposium, Oslo, Norway, June 24-26, 2015, Proceedings, Lecture Notes in Computer Science 9109, Springer 2015, ISBN 978-3-319-19248-2 [contents]

    Musab A. AlTurki , Omar Alzuhaibi:
Towards Formal Verification of Orchestration Computations Using the K Framework. 40-56

BibTex文件:

```
1 @inproceedings{DBLP:conf/fm/AlTurkiA15,  
2   author      = {Musab A. AlTurki and  
3                 Omar Alzuhaibi},  
4   editor      = {Nikolaj S. Bjørner and  
5                 Frank S. de Boer},  
6   title       = {Towards Formal Verification of  
7                 Orchestration Computations Using the  
8                 {\unicode{54594}} Framework},  
9   booktitle    = {{FM} 2015: Formal Methods - 20th  
10  International Symposium, Oslo, Norway,  
11                June 24-26, 2015, Proceedings},  
12  series       = {Lecture Notes in Computer Science},  
13  volume       = {9109},  
14  pages        = {40--56},  
15  publisher    = {Springer},  
16  year         = {2015},  
17  url          = {https://doi.org/10.1007/978-3-319-  
18                19249-9\_4},  
19  doi          = {10.1007/978-3-319-19249-9\_4},  
20  timestamp    = {Thu, 14 Apr 2022 20:26:16 +0200},  
21  biburl       =  
22  {https://dblp.org/rec/conf/fm/AlTurkiA15.bib},  
23  bibsource    = {dblp computer science bibliography,  
24                https://dblp.org}  
25 }
```

1 论文内容

- 该文章提出了一种为 **Orc** 编排语言设计形式化、可执行语义的方法。该方法的核心是利用 **K 框架 (K framework)** 来构建 Orc 的语义模型，从而能够对 Orc 程序的模型进行形式化的验证和分析。

2 背景知识

2.1 Orc语言 (Orchestration Language)

- 是一种为服务编排（编排，orchestration，很形象的用词，如同交响乐团指挥不同乐器声部演奏一样）设计的**形式化语言**（只能用来写伪代码，不是程序语言），提供了一个富有表现力的并发计算模型。Orc 有四个核心的组合子，它们构成了语言的表达能力：
 1. **并行 (Parallel)** `f | g` : 并发执行 f 和 g。并行不关心谁先获得结果，彼此之间完全独立
 2. **顺序 (Sequential)** `f >x> g` : 执行 f, f 每发布一个值 w, 就会创建一个 g 的新实例 (w 绑定到 x) 并执行。保证顺序
 3. **剪枝 (Pruning)** `f <x< g` : 并发执行 f 和 g, 但一旦 g 发布了第一个值, 就终止 g, 并将该值绑定到 f 中的 x。只关心第一个结果, 取得结果后立刻取消其他
 4. **否则 (Otherwise)** `f ; g` : 尝试执行 f, 只有当 f 终止且未发布任何值时, 才执行 g。备用方案
- 使用文中的例子来理解，比如需要从三个服务站点中得到当前黄金的价格，并将定价从 USD/Oz 转换成 Euro/gram. 下面的例子，并行访问三个服务站点，一旦拿到需要的数据就立刻剪枝并将结果传给 Converter 函数

```
1 | Converter(x, USD/Oz, EUR/gram) < x < (GoldSeek() | GoldPrice()  
  | Kitco())
```

- 文中的例子2，一个火灾报警函数，当检测到有火情时触发警报然后报告给最近的站点来提供工具灭火。是一个完全顺序执行的函数

```
1 | FireAlarm() > fireLoc > CalcNearestStation(fireLoc)  
2 | > station > Dispatch(station, fireLoc)
```

- 举个生活化的例子，假设想点一份披萨，手机里有三个外卖app: Meituan, Eleme和 JD. 希望尽快找到能送餐的餐厅，并且一旦下单成功，就想立即跟踪订单状态。这个过程可以用Orc表达式写成：

```

1 // 定义一个主流程：找到并下单，如果彻底失败，就换成搜寿司
2 FindAndOrderPizza ; SearchForOtherFood("Sushi")
3
4 // 定义“找到并下单”的具体过程
5 def FindAndOrderPizza =
6   // 1. 并行：同时在三个app上搜索“披萨”
7   // 2. 剪枝：只要任何一个app返回了第一个可行的餐厅offer，就立即采纳（绑定
   到变量offer），并终止其他两个App的搜索
8   ProcessOffer(offer) <offer< (Meituan("Pizza") | Eleme("Pizza")
   | JD("Pizza"))
9
10 // 定义“处理offer”的具体过程
11 def ProcessOffer(theOffer) =
12   // 3. 顺序：使用上一步得到的offer去下单，下单成功后会返回一个
   confirmation(确认信息)。然后，用这个确认信息去调用订单跟踪服务
13   PlaceOrder(theOffer) >confirmation> TrackDelivery(confirmation)

```

2.2 K framework

- K 框架 (K framework) 是一个用于形式化定义编程语言的框架，它包含了专门的句法符号和语义创新，旨在让语言的定义变得简洁和模块化。该框架基于上下文无关的项重写 (context-insensitive term rewriting)。

- K 框架建立在三个受现有语义框架启发的核心概念之上：

1. 计算结构 (Computational Structures / Computations):

- 计算是一个任务，将程序的抽象语法“扁平化”为一系列待执行的任务

2. 配置 (Configurations):

- 配置是程序执行过程中静态状态的一种表示
- K 框架将配置建模为一个可能嵌套的单元 (cell) 结构
- 这些单元是带标签的，代表了定义语义所需的基本组件，例如环境 (environments)、存储 (stores)、线程 (threads)、锁 (locks) 和栈 (stacks) 等

3. 规则 (Rules):

- 规则为语言的构造赋予语义。它们作用于配置（或配置的片段），将其转换为其他配置
- K 中有两种规则：
 - 结构性规则 (structural rules): 将配置的结构重排为一个行为等价的配置。
 - 计算性规则 (computational rules): 定义了不同配置之间外部可观察到的转换。

3 Orc在K中的语义

- 分为两个模块，语法模块和语义模块

3.1 语法模块

SYNTAX $Pgm ::= ExpDefs\ Exp$ SYNTAX $Arg ::= Val$ $Identifier$ SYNTAX $Call ::= ExpId(Params)$ $SiteId(Args)\ [strict(2)]$	SYNTAX $Exp ::= Arg$ $Call$ > $Exp > Param > Exp\ [right]$ > $Exp\ \ Exp\ [right]$ > $Exp < Param < Exp\ [left]$ > $Exp ; Exp\ [left]$
---	---

Fig. 3. Syntax of Orc as defined in $\mathbb{K}$

其实就是对文法Syntax的约定。

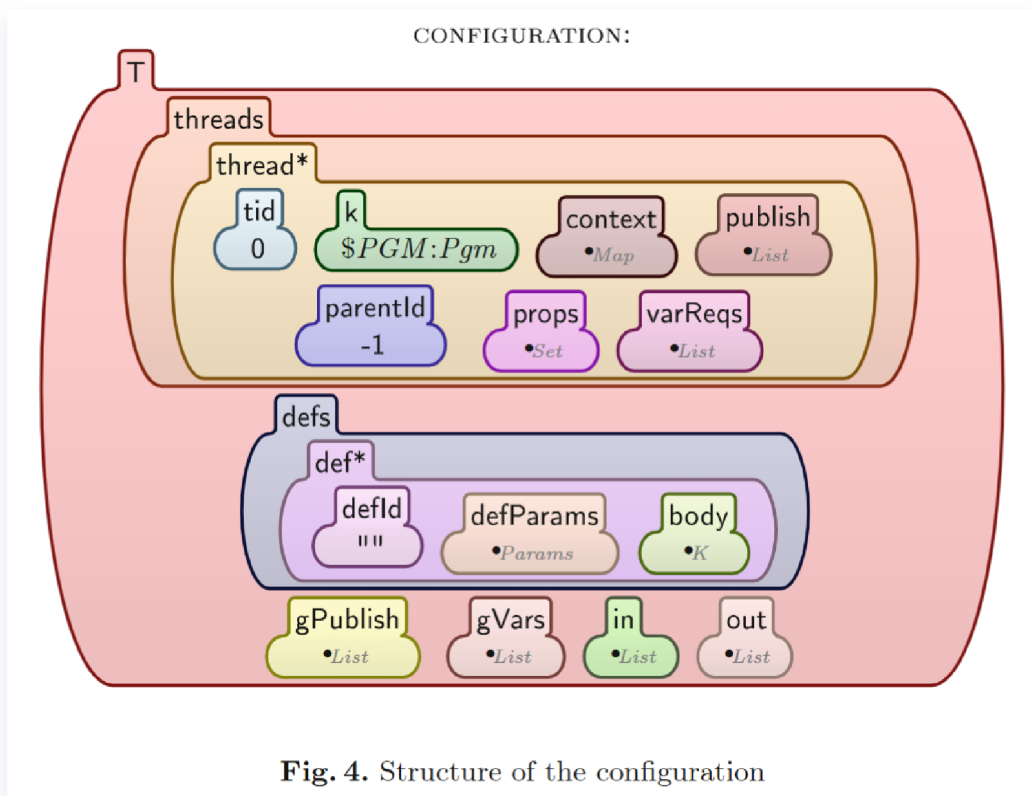
- 左边分别代表程序、参数、调用的语法；
- 右边定义了表达式的语法，并给出了优先级顺序：顺序>并行>剪枝>否则。另外，倾向于让简单的表达式先匹配，因此Arg和Call在前面
- 顺序和并行是右结合的，剪枝和否则是左结合的
- 严格性，strict(i)表示产生式的右手边第i个项在右侧项被评估之前必须匹配产生式

3.2 语义模块

就是把 Orc 语言的“执行规则”写成一组可执行的 **重写（rewrite）规则**，它由两大块组成：**配置结构（Configuration）** 和 **K 规则（K Rules）**

3.2.1 Configuration

K中的程序是嵌套的cell状（类似html语法），下图是Orc的初始配置：



转化成代码就是

```

1  <T>
2    <threads>...</threads>    // 线程池, 每个线程是一个 cell
3    <defs>...</defs>          // 程序开始的定义
4    <gPublish>...</gPublish>   // 全局发布的值
5    <gVars>...</gVars>        // 时钟、同步标志、环境变量
6    <in>...</in>               // 标准输入流
7    <out>...</out>             // 标准输出流
8  </T>

```

- **thread**: 可能有多个并发线程 (*), 每个线程内部有:
 - **k** : 计算单元, 存放当前要执行的 Orc 表达式
 - **context** : 该线程的变量映射
 - **publish** : 本地已发布值
 - **props** : 管理器标记 (如 **publishUp**)
 - **varReqs** : 变量请求, 供跨线程访问
- **defs**: 程序开头定义的全局函数/常量
- **gPublish**: 所有线程最终汇聚的“全局发布”结果
- **gVars**: 时钟 (**clock**)、时间上限标志 (**time.limit.reached**)、全局同步变量等
- **in/out**: 模拟 Orc 的 I/O 流

3.2.2 K Rules

K 规则就是在上述配置上做**重写**, $\Rightarrow$ 代表重写,

1. Parallel Combinator: PCM, parallel composition manager, 将两个并行进程分解开, 方框左下角的P是“PublishUP”标记, 表示要把它发布的值往父线程抛上去

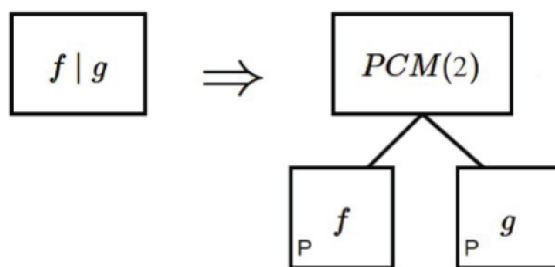


Fig. 5. Transformation rule of the parallel combinator

2. Sequential Combinator: SCM, short for Sequential Composition Manager, 右下角的 v 表示该线程已经发布了一个值。规则一是执行 f , 然后记录 g 的活动实例计数器 k (初始为0); 规则二在 f 发布 v 值之后执行, 重写成执行 g

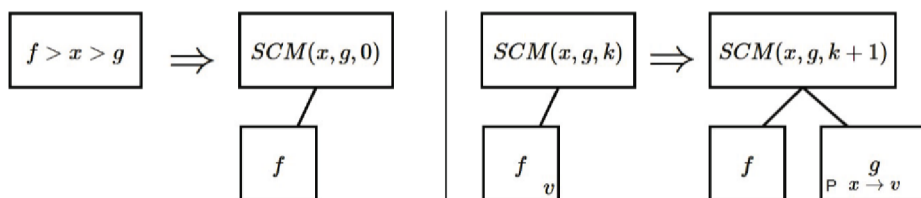


Fig. 6. Transformation rules of the sequential combinator

3. Pruning Combinator: PrCM (short for Pruning Composition Manager), 并发地启动 f 和 g ; 当 g 发布了它的第一个值 v 时, 触发规则二, g 终止, v 传递到线程, 并在其上下文中将 x 映射到 v

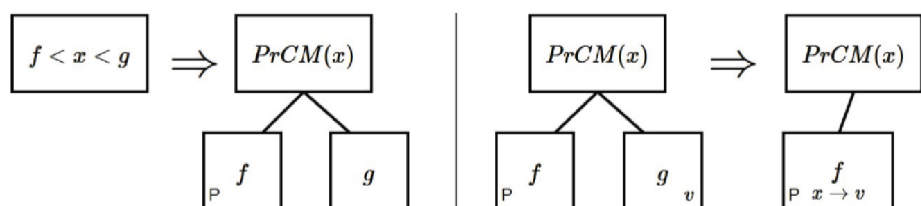


Fig. 7. Transformation rules of the pruning combinator

此处还给出了K代码结构：先匹配表达式，然后创建两个单独的线程，复制上下文，创建父子关系以及标记属性（F线程标记了是左边的子节点，以及要传递值到上层；G线程只标记了是右侧的子节点）

RULE PRUNING-PREP

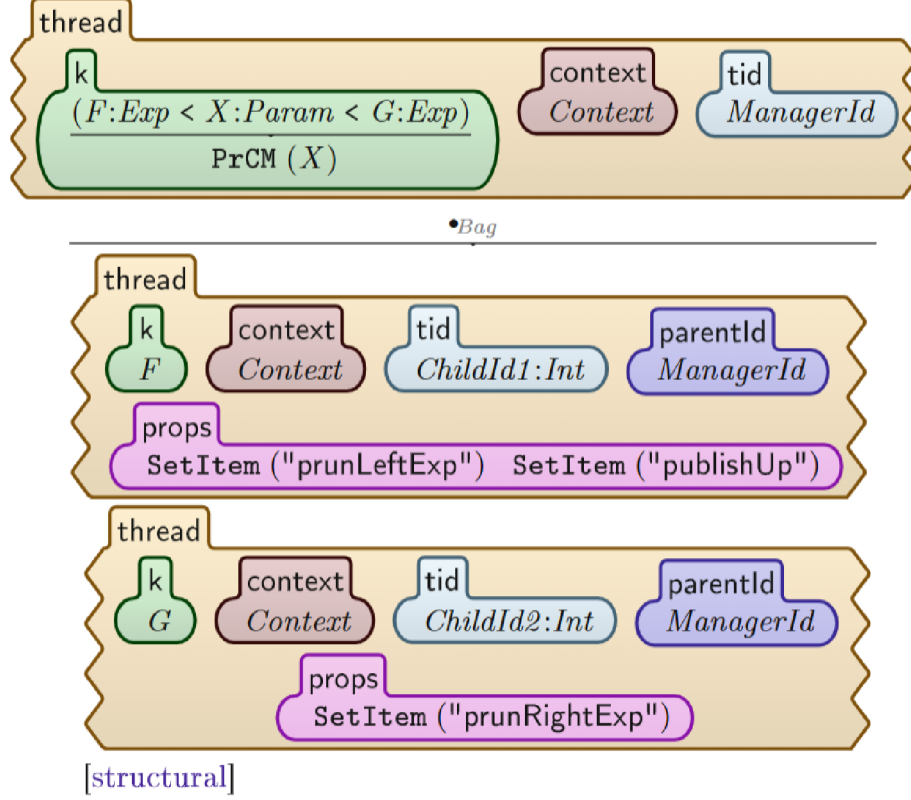


Fig. 10. First $\mathbb{K}$ rule of the Pruning Combinator

匹配成功时，对于 G 节点，将属性 `props` 中的“我是右侧节点”重写成“剪枝我”，便于后续的垃圾回收，同时 k 单元被清空表示不再有计算；对于 F 节点，获取到一个新的映射 $X \mapsto V$ ，将 G 发布的值 V 绑定到变量 X 上，并放入 F 的上文中，这样 F 在后续的计算中就可以使用 X 的值了

RULE PRUNING-PRUNE-RIGHT-AND-PASS-VALUE-TO-LEFT

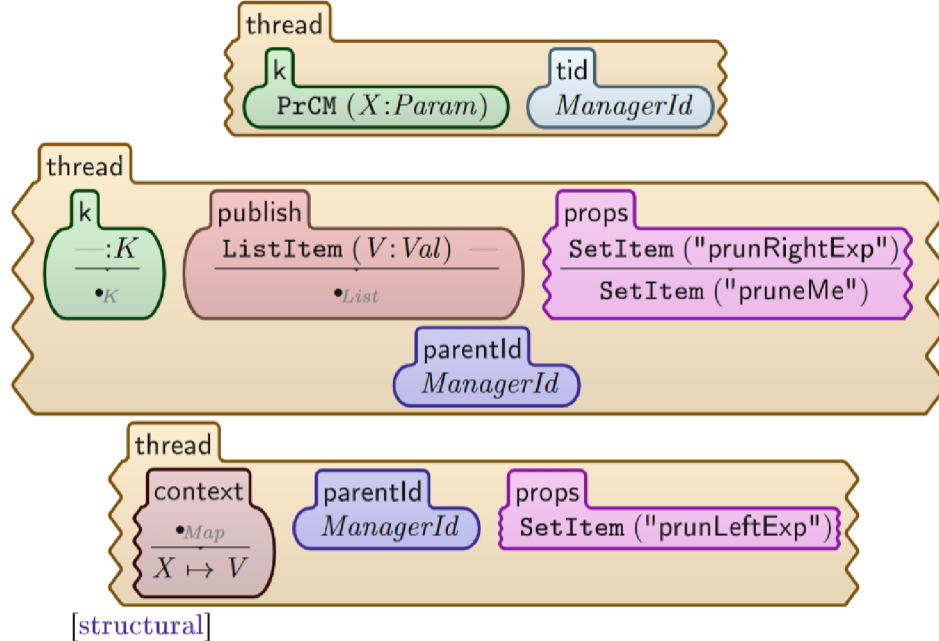


Fig. 11. Second $\mathbb{K}$ rule of the Pruning Combinator

4. Otherwise Combinator: OthCM (short for Otherwise Composition Manager), 规则一：执行f；规则二：当f发布第一个值，重写，丢弃g，并且f线程被赋予发布权限（标记P）以继续执行；规则三：f终止时，直接将 `f ; g` 表达式替换为表达式 `g`

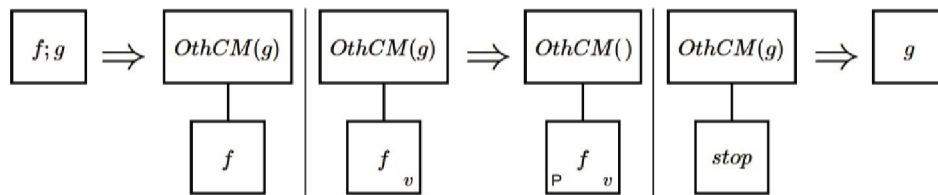


Fig. 8. Transformation rules of the otherwise combinator

4 Orc编排的形式化分析

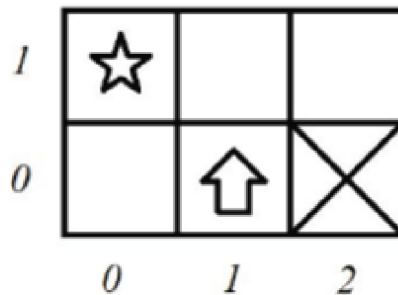


Fig. 12. Initial configuration of the robot environment

接下来使用一个简单的例子来对Orc编排进行形式化分析。模拟一个在有障碍物的房间里移动的机器人。定义以下操作：

1. `MoveFwd()` 将使机器人在其方向上移动一个区块的距离，需要 3 个时间单位
2. `turnRight()` 和 `turnLeft()` 将使机器人在同一区块上顺时针和逆时针旋转 90 度，需要 1 个时间单位
3. 在尝试向前移动时如果撞到障碍物，仍然会消耗 3 个时间单位，但会触发一个名为 `isBumperHit` 的标志，该标志将在下一个动作时重置

4.1 模拟

- 使用项目下的文件 `test_robot1.orc` 来做测试。其中大段的注释是作者当初调试的过程。
- 代码前半段定义了一些函数，我们重点关注函数 `CarefulStep(b)` 和函数 `CarefulStep4Ever(x)`。`CarefulStep(b)` 执行时先扫描前方是否有障碍物，然后通过顺序执行将变量 `isBlocked` 传入后方的表达式中；并行判断是否有障碍物；若有障碍物，顺序执行并行的左转和右转；若无障碍物，顺序执行向前走一步；`CarefulStep4Ever(x)` 是一个递归，不断执行 `CarefulStep(b)`。


```

1 CarefulStep(b) :=
2     bot.scan(x)                // 扫描前方
3     > isBlocked                // 顺序组合子
4     > ( if(isBlocked)          // 若有障碍，并行执行左转和右转
5         >> (bot.rotateRight(x)
6             | bot.rotateLeft(x)
7         )
8     | ifNot(isBlocked)        // 若无障碍，向前走一步
9         >> bot.stepFwd(x)
10    )
11
12 CarefulStep4Ever(b) :=
13     CarefulStep(b) >> CarefulStep4Ever(b)

```

- 根据代码70-76行，可以得知这是一个10x10的表格，且安放了众多障碍物。机器人名叫yourCow，初始位置是(2, 0), 方向是(0, 1)即朝向北。机器人并没有一个硬性的“终点”，而是执行 `CarefulStep4Ever(x)` 函数，一直在表格中安全的走下去。

```

1 bot.mapInit(<10,10>) >> (
2     bot.setObstacles(<1,1>,<1,5>,<2,2>,<2,4>,<2,8>,<3,7>,<3,4>,
3     <3,5>,<4,1>,<4,4>,<4,9>,<5,7>,<7,5>,<7,7>,<8,1>,<8,2>,<8,9>) >> (
4         bot.init("yourCow",<2,0>,<0,1>) > x > (
5             CarefulStep4Ever(x)
6         )
7     )

```

4.2 运行代码

4.2.1 环境配置

由于该论文完成于2015年，使用的K framework版本是3.5.2（而如今的K framework版本已经来到了7.x），其版本之古老使得直接在本机复现难度较大，需要支持其架构的Ubuntu-16.04，Java 8等。于是本人考虑使用docker进行镜像，`Dockfile` 内包含了所需的配置。代码的运行在Windows11的wsl系统下依靠Ubuntu-22.04完成。

```

1 docker pull ubuntu:16.04
2 mkdir orc-project
3 cd ~/orc-project
4 docker build -t orc-k:2015 .

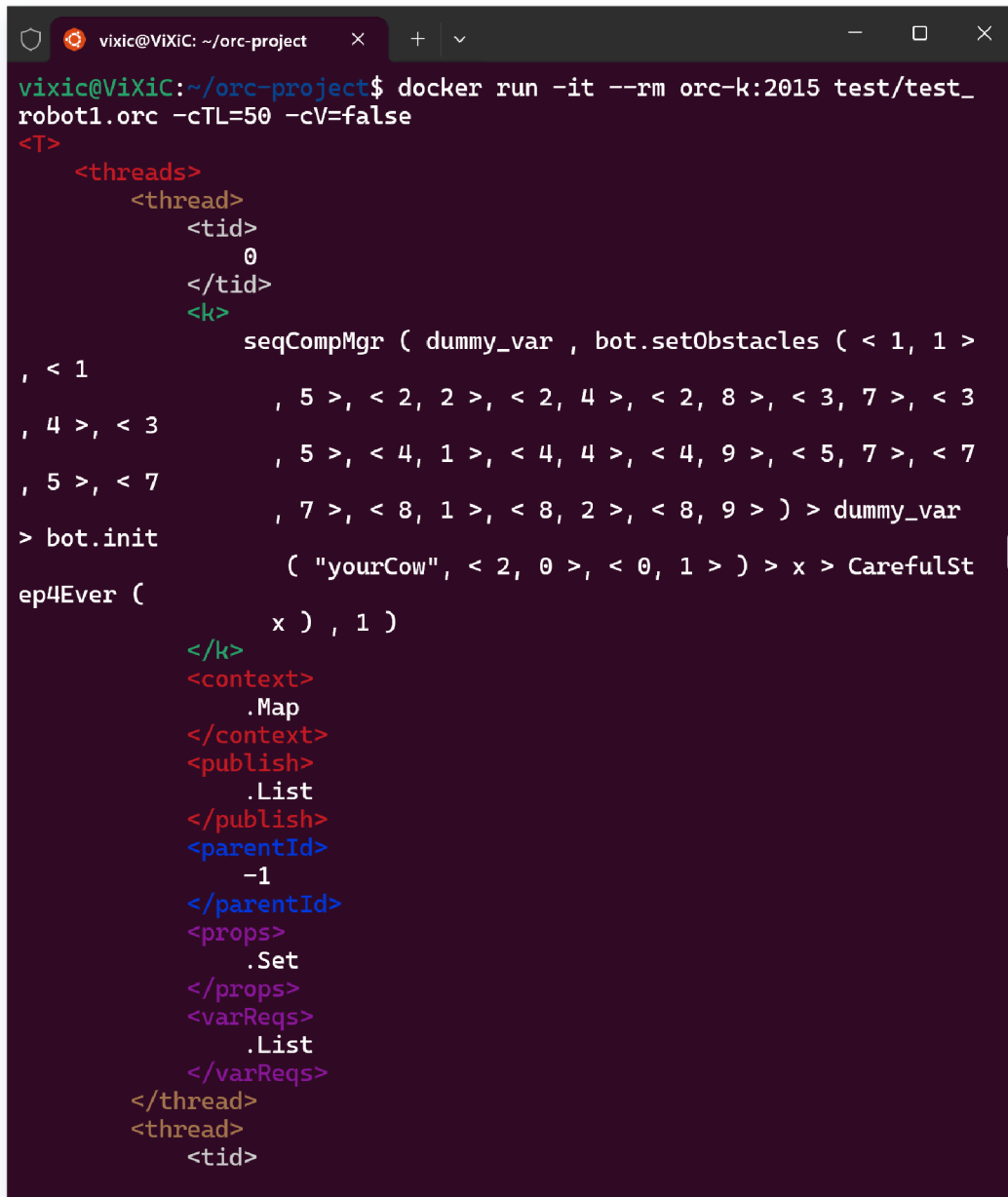
```

4.2.2 运行代码

使用指令

```
1 | docker run -it --rm orc-k:2015 test/test_robot1.orc -cTL=50 -  
   | cV=false
```

运行后在终端得到大段输出，以下是片段



```
vixic@ViXiC: ~/orc-project
vixic@ViXiC:~/orc-project$ docker run -it --rm orc-k:2015 test/test_
robot1.orc -cTL=50 -cV=false
<T>
  <threads>
    <thread>
      <tid>
        0
      </tid>
      <k>
        seqCompMgr ( dummy_var , bot.setObstacles ( < 1, 1 >
, < 1
, 5 >, < 2, 2 >, < 2, 4 >, < 2, 8 >, < 3, 7 >, < 3
, 4 >, < 3
, 5 >, < 4, 1 >, < 4, 4 >, < 4, 9 >, < 5, 7 >, < 7
, 5 >, < 7
, 7 >, < 8, 1 >, < 8, 2 >, < 8, 9 > ) > dummy_var
> bot.init
      ( "yourCow", < 2, 0 >, < 0, 1 > ) > x > CarefulSt
ep4Ever (
        x ) , 1 )
      </k>
    <context>
      .Map
    </context>
    <publish>
      .List
    </publish>
    <parentId>
      -1
    </parentId>
    <props>
      .Set
    </props>
    <varReqs>
      .List
    </varReqs>
  </thread>
  <thread>
    <tid>
```

...

```
vixic@ViXiC: ~/orc-project
    ListItem ( 8 )
    ListItem ( 2 ) )
  ListItem (
    ListItem ( 8 )
    ListItem ( 9 ) )
"bot.map_size" |->
  ListItem ( 10 )
  ListItem ( 10 )
"bot.yourCow.block_indicator" |->
  ListItem ( 0 )
  ListItem ( 5 )
"bot.yourCow.bot_lock" |-> true
"bot.yourCow.direction" |->
  ListItem ( 0 )
  ListItem ( 1 )
"bot.yourCow.is_bumper_hit" |-> false
"bot.yourCow.is_facing_wall" |-> false
"bot.yourCow.position" |->
  ListItem ( 0 )
  ListItem ( 4 )
"count" |-> 0
"publishCount" |-> 47
"time.MTE" |-> 2
"time.clock" |-> 50
"time.limit.reached" |-> true
"time.ticked" |-> 0
</gVars>
<in>
  ListItem ( #buffer ( " " ) )
  ListItem ( #istream ( 0 ) )
</in>
<out>
  ListItem ( #ostream ( 1 ) )
  ListItem ( #buffer ( " " ) )
</out>
<verbose>
  false
</verbose>
</T>
```

输出的格式遵循一定的格式，完整的输出见文件 `test_robot1.ansi`。运行完成后当前的状态参照输出中的 `<gVars>` 段，先是列出了所有障碍物的位置，然后描述了当前机器人的方向、位置和状态等。可以看到

```
1  <gVars>
2      "bot.map_obstacles" |→
3          ListItem (...) // 障碍物坐标列表
4      "bot.map_size" |→
5          ListItem ( 10 )
6          ListItem ( 10 )
7      "bot.yourCow.block_indicator" |→ // 上一次扫描前方是
    否有阻碍
8          ListItem ( 0 )
9          ListItem ( 5 )
10     "bot.yourCow.bot_lock" |→ true // 一个互斥锁，保证
    机器人动作的串行化
11     "bot.yourCow.direction" |→
12         ListItem ( 0 )
```

```

13         ListItem ( 1 )                                // 朝向 (0,1) 北向
14         "bot.yourCow.is_bumper_hit" ↦ false          // 没撞到障碍
15         "bot.yourCow.is_facing_wall" ↦ false         // 没面朝墙
16         "bot.yourCow.position" ↦
17         ListItem ( 0 )
18         ListItem ( 4 )                                // 当前位置 (0,4)
19         "count" ↦ 0
20         "publishCount" ↦ 47
21         "time.MTE" ↦ 2
22         "time.clock" ↦ 50                             // 模拟跑到 50 个
tick
23         "time.limit.reached" ↦ true                  // 因为 ` -cTL=50 `
限制了只能跑50个tick
24         "time.ticked" ↦ 0
25     </gVars>

```

可以看到这个模型下机器人的行为正常可靠。作者在第4部分最后提到,

"We demonstrated the potential of exploiting K's state search capabilities for purposes of formal verification. Other methods that K provides such as Maude's LTL model checker and Maude's proof environment are sure to deliver more in-depth verification."

更深度的验证可以借助Maude工具完成

5 总结与经验

1. 学习了对并行任务进行编排的语言Orc，以及将其四种组合子(combinator)转化成K framework中的四类管理器（PCM/SCM/PrCM/OthCM）的规则，学到了一种对orc语言进行形式化的方案；
2. 进行转化的过程中运用到了编译原理的知识，和orc的形式化息息相关。同时也了解到并行任务的形式化（在当时）仍然处于探索阶段。通过搜索得知，当前并行任务形式化编排的可组合性、可验证性和可扩展性上超越早期 Orc-K 案例；

结论：

是的，现在确实有比论文发表时更成熟的方案来对并行任务进行形式化编排和验证。这种成熟体现在：

1. **核心框架** (K, TLA+, UPPAAL, Coq, Isabelle, FDR4 等) 本身的持续发展和稳定化。
2. **工具链的完善和用户体验的改进** (尽管学习曲线仍然存在)。
3. **验证方法论的进步** (组合验证、概率验证、实时验证)。
4. **更广泛、更复杂的工业应用场景得到实际验证。**
5. **与其他工程实践** (如 DSL 设计、代码生成、运行时监控) 的更好集成。

虽然 Orc + K 提出的基于并发重写的真并发语义方法在理论上具有吸引力，并且 K 框架持续活跃，但在并行任务形式化编排的实践成熟度和广泛采纳方面，其他方案如 TLA+ (用于分布式算法/系统)、UPPAAL (用于实时系统)、成熟的进程代数工具 (如 FDR4 for CSP) 以及定理证明器 (Coq/Isabelle 用于最高保证) 目前代表了更为主流和公认的“成熟”方案。领域特定语言与形式化方法的结合也是一个积极的发展方向。该领域仍在快速发展中，尤其是在应对云原生、大规模分布式和智能边缘系统带来的新挑战方面。

3. 由于考虑不周，选择了一篇2015年的论文，导致环境配置成了比较大的问题，期间历经dockerfile的多次检查和尝试，排查和搜索接近8小时后终于成功运行起来代码；

5 总结与经验